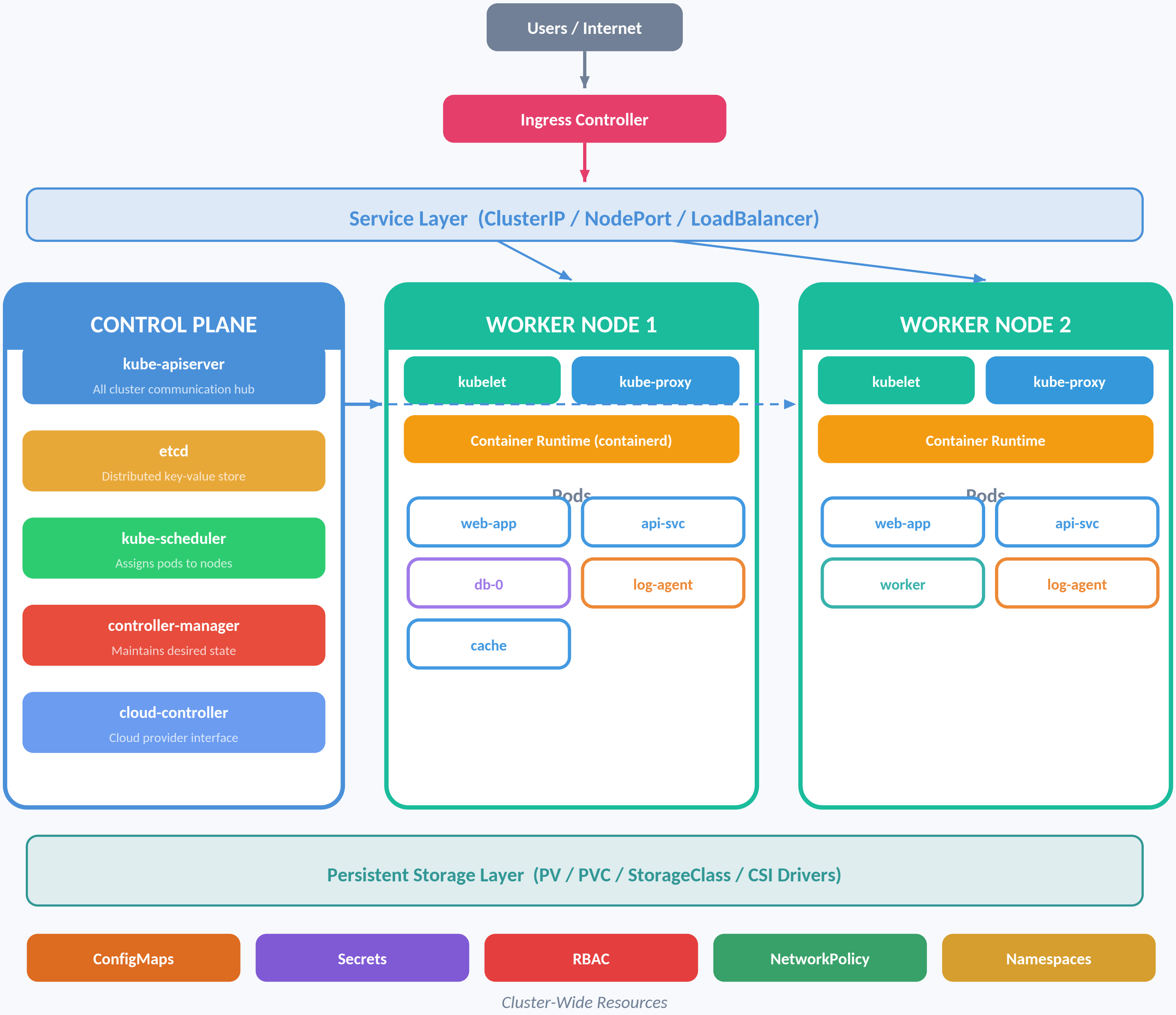
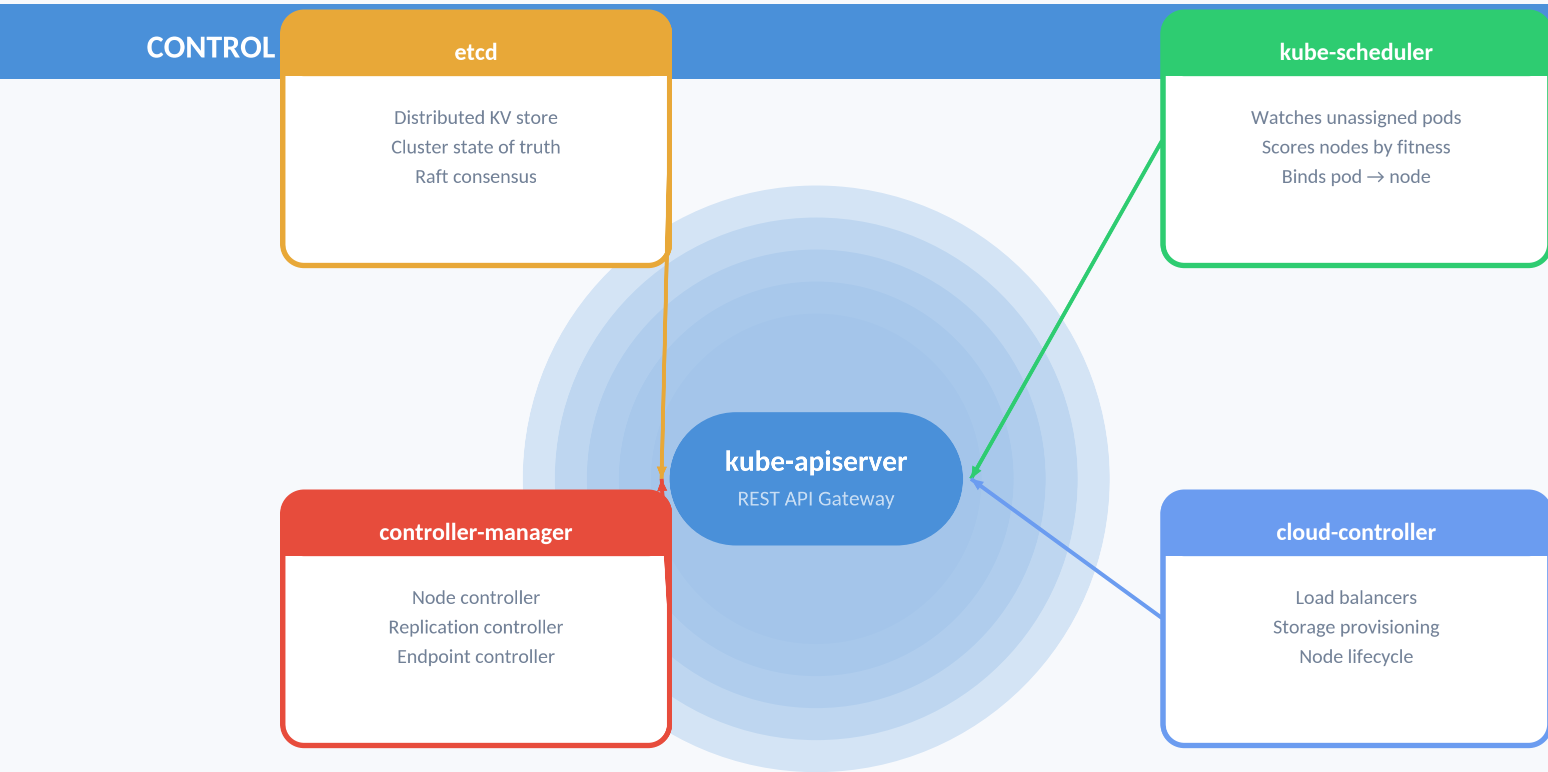
Kubernetes

Visual Architecture Reference

Every Core Concept — One Visual at a Time

Control Plane · Workloads · Networking · Storage · Config · Security

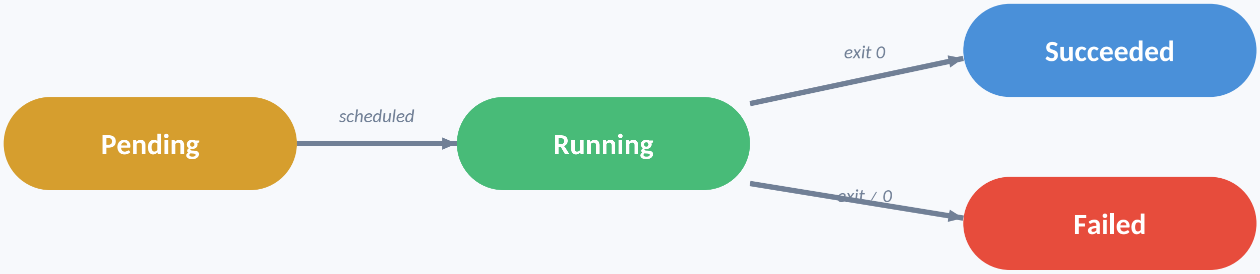

Users / Internet
Ingress Controller
Service Layer (ClusterIP / NodePort / LoadBalancer)
CONTROL PLANE
kube-apiserver
All cluster communication hub
etcd
Distributed key-value store
kube-scheduler
Assigns pods to nodes
controller-manager
Maintains desired state
cloud-controller
Cloud provider interface
WORKER NODE 1
kubelet
kube-proxy
Container Runtime (containerd)
Pods
web-app
api-svc
db-0
log-agent
cache
WORKER NODE 2
kubelet
kube-proxy
Container Runtime
Pods
web-app
api-svc
worker
log-agent
Persistent Storage Layer (PV / PVC / StorageClass / CSI Drivers)
ConfigMaps
Secrets
RBAC
NetworkPolicy
Namespaces
Cluster-Wide Resources



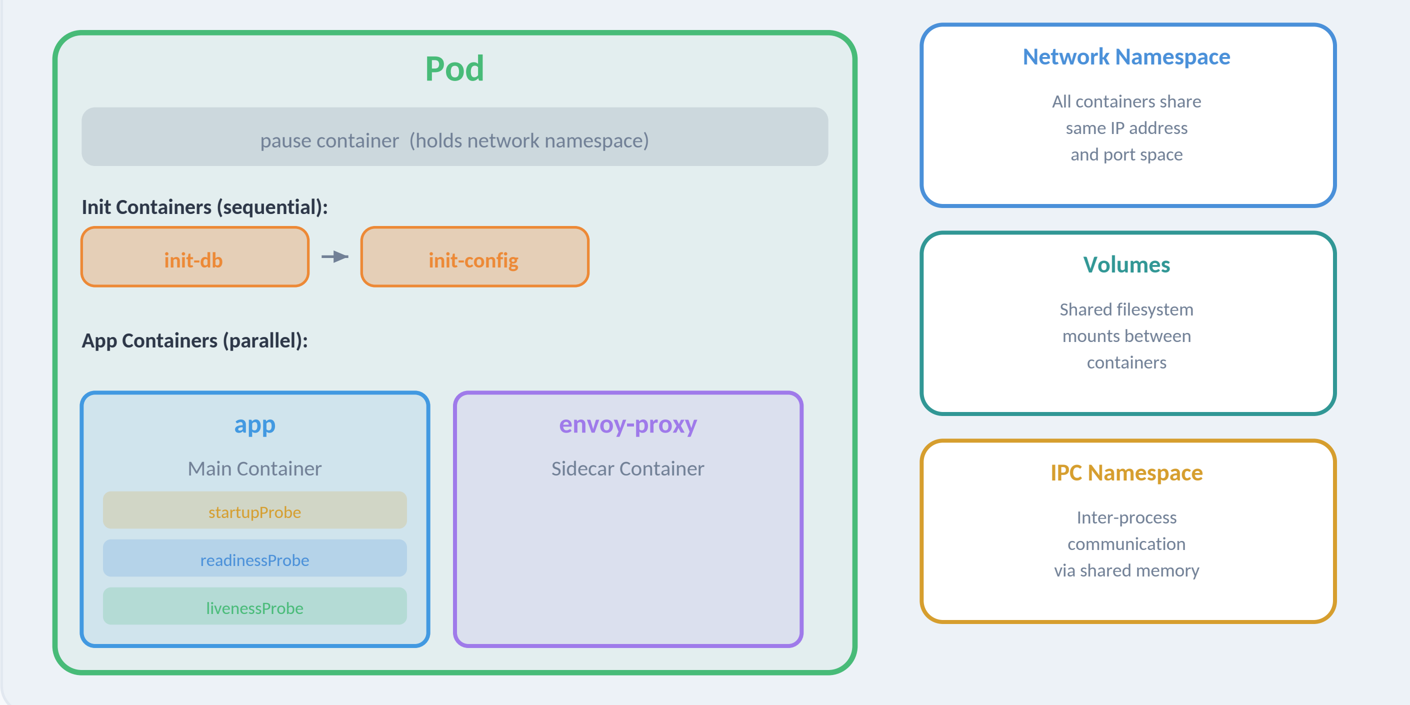
Request Flow: kubectl apply → Running Pod

- All communication goes through the API Server — it is the single point of access
- etcd is the ONLY stateful component — losing it means losing cluster state
- Scheduler and Controller Manager are stateless and leader-elected for HA


WORKER NODE (Linux Host / VM / Bare Metal)
Node Agents
kubelet
Pod lifecycle manager
kube-proxy
iptables / IPVS rules
containerd / CRI-O
Container runtime (CRI)
Pods (Scheduled Workloads)
Pod: web-app-7d8f9b
Containers:
nginx:1.25
Main Container
Port: 80
CPU: 100m Mem: 128Mi
fluentd
Sidecar
Log shipping
Shared:
IP: 10.0.1.5
Volume: data
ConfigMap
container
log-agent
container
backup-job
container
Node Resources
CPU
8 cores
4.2 used
Memory
32 Gi
18 Gi used
Pods
110 max
47 running
Storage
500 Gi
180 Gi used



Pod Internal Architecture



Pod QoS Classes

- **Guaranteed: requests == limits**
- **Burstable: requests < limits**
- **BestEffort: no requests/limits**

WORKLOAD CONTROLLERS — Managing Pods at Scale

ReplicaSet
Deployment

r0 r1 r2

- Stateless apps
- Rolling updates
- Rollback support
- Scale horizontally

Use for:

web servers, APIs microservices

Ordered pods
StatefulSet

-0 -1 -2

- Stable network IDs
- Persistent storage per pod
- Ordered deploy/scale
- Stable DNS: pod-0.svc

Use for:

databases, Kafka, ZooKeeper

1 per node
DaemonSet

- Runs on ALL nodes
- Auto-added to new nodes
- Node-level agents
- Tolerates taints

Use for:

log collectors, monitoring, CNI

Run to completion
Job

→

- One-off tasks
- Retry on failure
- Parallelism support
- completions count

Use for:

migrations, batch processing

Scheduled
CronJob

- Cron syntax schedule
- Creates Job objects
- Concurrency policy
- History limits

Use for:

backups, reports, cleanup

ClusterIP

Internal only
Stable virtual IP
Default type

Use for:

cluster-internal comms

NodePort

ClusterIP +
Port on every node
Range: 30000-32767

Use for:

dev/testing external access

LoadBalancer

NodePort +
Cloud LB provisioned
External IP assigned

Use for:

production external traffic

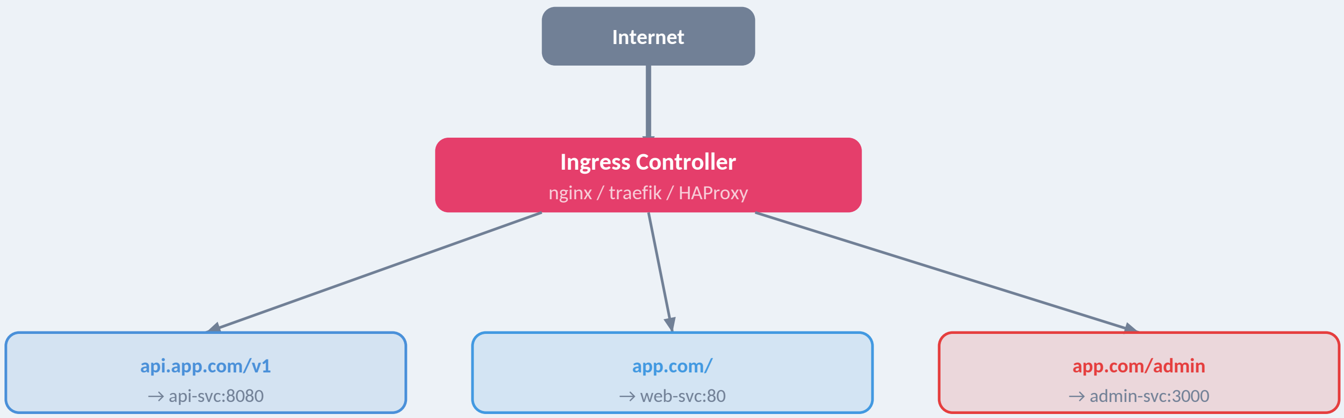
ExternalName

CNAME alias
No proxy/selector
DNS-level redirect

Use for:

external service mapping

Ingress Routing — L7 Load Balancing



Cluster DNS (CoreDNS)

Service:
my-svc.my-ns.svc.cluster.local

Pod:
10-0-1-5.my-ns.pod.cluster.local

Headless:
pod-0.my-svc.my-ns.svc.cluster.local

STORAGE — Persistent Data Architecture

Pod

volumeMounts: [{name: data, mountPath: /data}]

binds to

PersistentVolumeClaim (PVC)

Request: 10Gi, ReadWriteOnce, storageClass: fast

binds to

StorageClass

Provisioner: kubernetes.io/aws-ebs, type: gp3

binds to

PersistentVolume (PV)

Capacity: 10Gi, NFS/EBS/GCE-PD/Ceph

binds to

Physical Storage

AWS EBS / GCE PD / NFS / Ceph / Local

Access Modes

RWO	ROX	RWX	RWOP
ReadWriteOnce	ReadOnlyMany	ReadWriteMany	ReadWriteOncePod
Single node R/W	*Multi-node read*	*Multi-node R/W*	*Single pod R/W*

Reclaim Policies

- **Retain: Keep PV data after PVC deleted**
- **Delete: Remove PV and backing storage**
- **Recycle: Deprecated — scrub and reuse**

ConfigMap

- Non-sensitive configuration data
- Key-value pairs or file content
- Mounted as files or env vars
- NOT encrypted — plain text

e.g. DB_HOST=postgres.prod, APP_PORT=8080

Secret

- Sensitive data (passwords, tokens, keys)
- Base64-encoded (NOT encrypted!)
- Types: Opaque, TLS, Docker registry
- Encrypted at rest with --encryption-provider

Always encrypt at rest + use external vault

RBAC

Subject (User/Group/SA) → **RoleBinding** (Grants role) → **Role/ClusterRole** (Permissions)

Verbs: get, list, watch, create, update, patch, delete

Role = namespace-scoped | ClusterRole = cluster-wide

RoleBinding = namespace | ClusterRoleBinding = cluster

ServiceAccount:

Identity for pods — auto-mounted token at /var/run/secrets/

NetworkPolicy

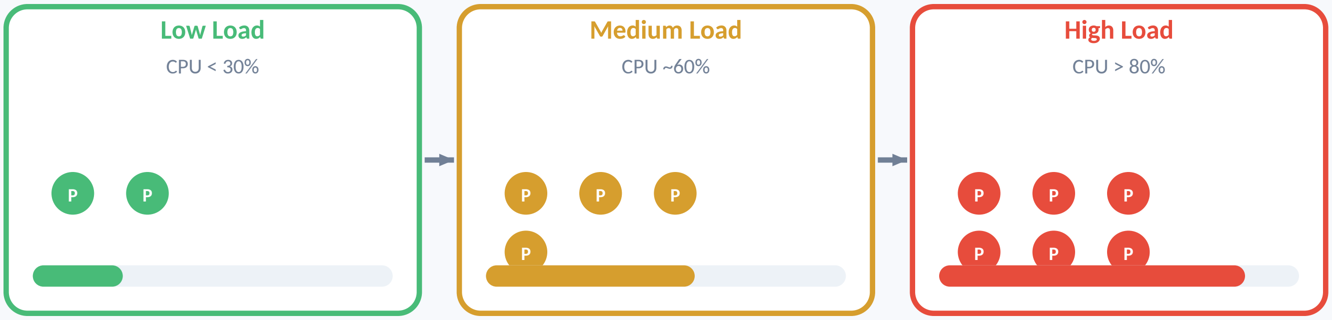
- Default: all traffic allowed (no isolation)
- podSelector: which pods the policy applies to
- ingress rules: who can SEND traffic IN
- egress rules: where pods can SEND traffic OUT
- Requires CNI plugin (Calico, Cilium, Weave)

Namespaces — Logical Cluster Partitioning

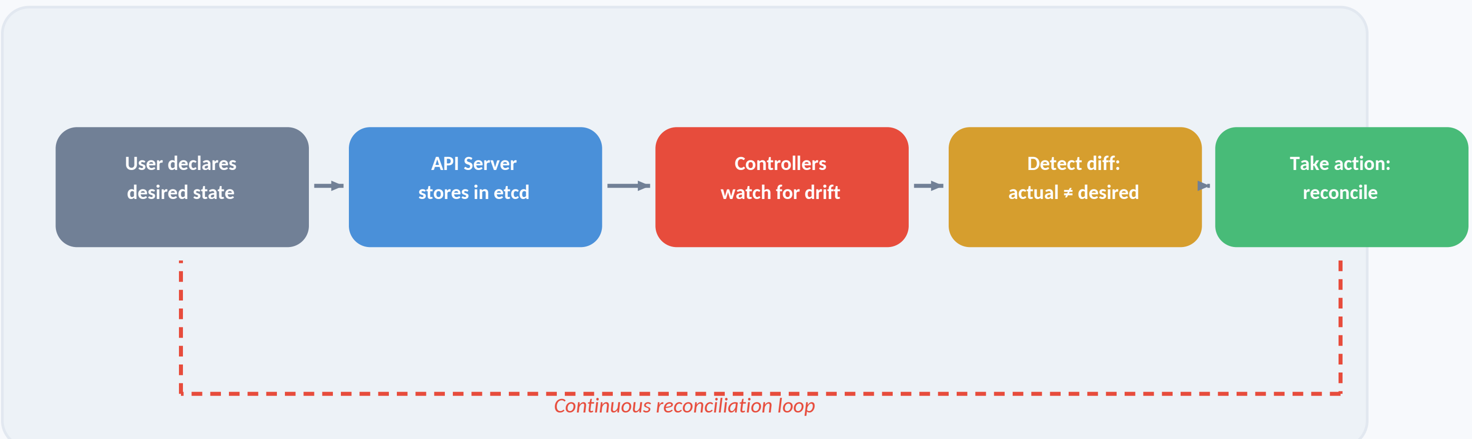
Resource isolation • RBAC scoping • ResourceQuotas • LimitRanges • Network boundaries

SCALING & SELF-HEALING — Resilience Built In

Horizontal Pod Autoscaler (HPA)



Self-Healing: Desired State Reconciliation



Self-Healing in Action

- **Pod crash** → Controller detects, creates replacement
- **Node failure** → Pods rescheduled to healthy nodes
- **Failed health check** → Pod restarted (liveness) or removed from svc (readiness)
- **Replica count drift** → ReplicaSet scales up/down to match desired count

Helm — Package manager for Kubernetes

Chart	Package of K8s manifests + templates
Release	Installed instance of a chart
Repository	Collection of charts (like apt/yum)
Values	Customizable parameters for templates

Operators — Custom controllers for complex applications

CRD (Custom Resource)	Extend K8s API with domain-specific objects
Controller	Watches CRDs, automates lifecycle (backup, failover)
Examples	Percona Operator, Prometheus Operator, Cert-Manager

kubectl Quick Reference

`kubectl get pods -A`	List all pods in all namespaces
`kubectl describe pod <name>`	Detailed pod info + events
`kubectl logs <pod> -c <container>`	Container logs
`kubectl exec -it <pod> -- /bin/sh`	Shell into container
`kubectl apply -f manifest.yaml`	Create/update resources
`kubectl delete -f manifest.yaml`	Delete resources
`kubectl scale deploy <name> --replicas=5`	Scale deployment
`kubectl rollout status deploy/<name>`	Watch rollout progress
`kubectl rollout undo deploy/<name>`	Rollback deployment
`kubectl top nodes / pods`	Resource usage
`kubectl get events --sort-by='.lastTimestamp'`	Recent cluster events
`kubectl port-forward svc/<name> 8080:80`	Local port forwarding